Strategic Implementation Framework for Multimodal AI-Driven Fraud Detection and Prevention on the Databricks Intelligence Platform

The global insurance and financial services landscape is currently navigating a period of unprecedented risk characterized by the democratization of generative artificial intelligence, which has significantly lowered the barrier to entry for high-fidelity document and media forgery. Traditional fraud detection mechanisms, predominantly reliant on deterministic business rules and structured data analysis, are increasingly proving inadequate against the sophistication of AI-generated synthetic media, manipulated identity documents, and staged visual evidence. In response to this shifting threat landscape, the collaboration between industry leaders such as Chubb and technology providers like Databricks emphasizes the necessity of a transition toward multimodal, data-centric AI platforms. This report provides an exhaustive analysis and an expert-level implementation framework for an integrated fraud identification system targeting documents, images, and videos. By leveraging the Databricks Intelligence Platform, organizations can synthesize disparate data modalities into a unified security posture that enhances operational effectiveness and safeguards financial integrity.

The Evolution of AI-Driven Fraud in the Multimodal Ecosystem

The contemporary insurance claim process generates a dense multimodal data ecosystem, where a single incident report typically comprises structured application forms, unstructured narratives from adjusters and claimants, photographic evidence of damage, and increasingly, video recordings such as dashcam or surveillance footage.[1] Historically, these data types were analyzed in isolation, creating siloes that sophisticated fraudsters could exploit. For example, a claimant might submit a narrative that is internally consistent but which contradicts the hidden metadata or visual artifacts present in the accompanying photographic evidence.[1] The "next frontier" in fraud mitigation involves the development of AI systems capable of fusing and interpreting these disparate modalities simultaneously to identify non-linear patterns of deception that are invisible to single-source systems.[1]

The economic impetus for this transition is clear; insurance fraud costs the industry billions of dollars annually, and traditional rule-based systems often generate high false-positive rates that disrupt customer satisfaction and increase administrative overhead.[2] The adoption of multimodal AI frameworks allows for a paradigm shift from reactive detection to proactive, intelligent fraud management.[1]

Fraud Modality	Traditional Detection Method	AI-Driven Multimodal Approach
Structured Data	Rule-based flags (e.g., claim within 14 days of policy)	XGBoost/ML models identifying non-linear risk patterns [1]
Unstructured Text	Manual review by adjusters	LLM-based narrative consistency and deceptive language analysis [1]
Static Images	Visual inspection for obvious edits	CNN-based anomaly detection and EXIF metadata forensics [1]
Video Evidence	Spot-checking key moments	Automated frame sampling, temporal analysis, and pHash deduplication [6]

Evaluation of the Proposed Hackathon Solution

The proposed solution by the team—utilizing Unity Catalog Volumes for storage, AI Playground for initial scoring, and a Databricks App for the user interface—provides a robust foundation for a 24-hour hackathon project. By focusing on the Databricks ecosystem, the team maximizes the utility of managed services, thereby reducing the time spent on infrastructure management and allowing for a greater focus on the "intelligence" of the application.[8]
The decision to utilize Unity Catalog Volumes for unstructured data (JPG, PNG) is technically sound, as Volumes are specifically designed for the governance and management of non-tabular data, providing a governed path to cloud object storage while maintaining compatibility with standard Python libraries.[10] Furthermore, the use of AI Playground for interactive scoring allows for rapid prototyping against state-of-the-art foundation models like Llama-3-Vision or GPT-4o without the overhead of manual endpoint configuration.[12] However, to elevate this solution to a professional grade suitable for real-world deployment, several critical enhancements in forensic analysis, video processing, and agentic reasoning are required.

Advanced Technical Improvements and Forensic Strategies

To move beyond simple image scoring, the system must incorporate a multi-layered forensic approach that analyzes both the "pixel-level" data and the "metadata" associated with each file.

Image Metadata and Digital Forensics

Digital images contain a wealth of hidden information beyond visible pixels, known as Exchangeable Image File Format (EXIF) metadata.[14] In a fraud detection context, EXIF data is a primary source of evidence for verifying the authenticity of submitted photographs.[14] A

professional-grade system should automate the extraction and analysis of several key metadata tags to identify discrepancies.

EXIF Tag	Forensic Relevance	Fraud Indicator
Software	Identifies the tool used to save the image	Presence of "Adobe Photoshop", "GIMP", or "Canva" [14]
DateTimeOriginal	The exact timestamp of the photo capture	Date does not align with the reported incident or loss date [14]
Make/Model	The hardware used to take the photo	Claimed device (e.g., iPhone 15) does not match EXIF data [5]
GPS Info	Coordinates of the image capture	Location is geographically inconsistent with the reported accident site [16]
ImageWidth/Height	Native resolution of the sensor	Resolution doesn't match the standard output of the detected device model

While the team's initial plan focuses on "scoring" images, the integration of a dedicated forensic script—using libraries such as exif or exiftool—within the Databricks environment provides an objective "red flag" system that complements the subjective analysis of vision models.[14] Furthermore, the absence of EXIF metadata in a context where it is typically expected (such as a direct upload from a smartphone) is itself a significant indicator that the image may have been scraped from the internet or processed through a manipulation tool that stripped the metadata.[5]

Multimodal Consistency Analysis

The true power of the Databricks platform lies in its ability to perform "cross-modal" checks. An improved implementation should not only score an image but also compare that image against the textual claim description. This can be achieved using the ai_query function to invoke a vision-language model (VLM) with a prompt that explicitly asks for consistency verification.[13] For instance, if a claimant reports a "low-speed rear-end collision" but the submitted images show significant front-end damage or structural warping inconsistent with low speed, the AI can flag this as a high-risk inconsistency.[1] This narrative-to-visual mapping is a critical real-world requirement that goes beyond the capabilities of unimodal systems.[1]

Video Processing Framework and Performance Optimization

Video evidence, such as dashcam footage or remote video inspections, offers higher fidelity than static images but presents significant computational challenges.[18] In a 24-hour hackathon timeline, the most effective strategy for video identification is intelligent frame sampling rather than exhaustive frame-by-frame analysis.

The use of the Decord library is recommended for its high-performance video reading capabilities, which allow for seeking frames by index without the need for sequential decoding—a process that is significantly faster than standard OpenCV loops.[6] To ensure high-quality results while minimizing token costs, the system should implement a "gating" mechanism. This involves extracting frames at a set interval (e.g., every 1 second), calculating perceptual hashes (pHash) to deduplicate near-identical frames, and using a lightweight blur detection algorithm to filter out unusable samples.[7]
For large-scale video processing, the implementation should leverage Spark User-Defined Functions (UDFs) to distribute the frame extraction across the Databricks cluster.[13] To further optimize performance, the team should enable Apache Arrow for the UDFs, which accelerates data (de)serialization between the JVM and Python processes.[22]
\
By utilizing Arrow-optimized Python UDFs, the system can handle large datasets of video frames with the efficiency required for near-real-time authentication.[22]

Real-World Applications and Targeted Use Cases

To demonstrate the maximum impact to judges and stakeholders, the team should target specific real-world problems where multimodal authentication provides a clear Return on Investment (ROI).

Insurance Claims Automation and "Low-Touch" Processing

In the insurance sector, the primary goal of AI is to enable "straight-through processing" (STP) for legitimate claims while identifying high-risk cases for manual investigation.[23] By automating the verification of damage photos against claim narratives, insurers can reduce settlement times from weeks to minutes, leading to a reported 70% increase in processing efficiency.[24]

Application	Specific Fraud Detected	Business Impact
Auto Insurance	Fabricated damage, staged accidents, pre-existing condition claims	Reduced loss ratio, lower investigation costs [19]
Property/Home	Exaggerated destruction, AI-generated "total loss" images	Prevention of large-scale indemnity leakage [18]
Health/Disability	Falsified X-rays, manipulated medical billing documents	Mitigates systemic medical fraud [19]

Banking and Identity Verification (KYC)

Financial institutions are increasingly vulnerable to "Deepfake" identity fraud, where synthetic videos or photoshopped ID cards are used to open fraudulent accounts.[15] A multimodal system can verify the authenticity of an ID card by performing OCR on the text, checking for font inconsistencies, and comparing the portrait photo against a live "selfie" video using biometric

matching and liveness detection.[15]

Supply Chain and Global Trade

In global trade, fraudulent "Bills of Lading" or manipulated shipping invoices are used to hide grey-market goods or facilitate money laundering. A Databricks-based solution can ingest scanned shipping documents using ai_parse_document and cross-reference them with visual evidence of the cargo at port, identifying discrepancies in weight, volume, or labeling that suggest documentation fraud.[11]

Detailed Implementation Plan for a 24-Hour Hackathon

Success in a high-pressure hackathon environment requires a tiered approach that prioritizes a "Minimum Viable Product" (MVP) followed by incremental enhancements.

Phase 1: Environment Setup and Data Ingestion (Hours 1–4)

The foundation of the project is the creation of a governed data landing zone. The team should initialize a new Catalog and Schema in Unity Catalog to store all project assets.[10]

1. **Volume Creation**: Create a managed Unity Catalog Volume named raw_evidence. This volume will host the uploaded images and videos, allowing them to be referenced via standard file paths such as /Volumes/catalog/schema/raw_evidence/file.jpg.[10]
2. **Sample Data Ingestion**: Rather than manually sourcing data, use Databricks' pre-built datasets or generative scripts to populate the volume with a mix of "clean" and "fraudulent" samples.[10]
3. **Governance Configuration**: Assign the necessary READ VOLUME and WRITE VOLUME privileges to the cluster or service principal to ensure seamless data flow.[27]

Phase 2: Building the Forensic Pipeline (Hours 5–10)

This phase focuses on the "First Pass" analysis using deterministic rules and basic AI.

1. **Metadata Script**: Develop a Python notebook that iterates through the files in the Volume and extracts EXIF data. Store this metadata in a Delta table named metadata_forensics.[5]
2. **Document Extraction**: Apply the ai_parse_document function to any PDF invoices or claim forms to extract key-value pairs (e.g., claim amount, date, location) into a structured Delta table.[11]
3. **Initial Scoring**: Use the AI Playground or a dedicated notebook to call a foundation model like Llama-3-Vision. Pass the image along with a specific prompt: "Analyze this vehicle damage. Does it show signs of digital manipulation or inconsistent lighting?".[4]

Phase 3: Video Integration and Batch Scoring (Hours 11–16)

The project expands into the temporal dimension by processing video files.

1. **Spark Video UDF**: Implement the frame sampling logic using Decord wrapped in a Spark UDF. This allows for parallel processing of multiple video files across the cluster's worker nodes.[6]
2. **Batch Inference Pipeline**: Use the mlflow.pyfunc.spark_udf to wrap the image scoring model. Apply this UDF to the extracted video frames to generate a "suspicion timeline" for each video file.[13]
3. **Vector Search for "Known Fakes"**: (Pro Tip) Embed the images using databricks-gte-large-en and store them in a Vector Search index. When a new image is uploaded, check if it is "too similar" to known fraudulent images found on the internet.[17]

Phase 4: Developing the User Interface (Hours 17–21)

The "Databricks App" (Streamlit) provides the front-end for the solution.

1. **File Uploader**: Implement st.file_uploader to allow users to submit images, docs, or videos.[29]
2. **Results Dashboard**: Use st.dataframe to show the forensic scores and st.image to highlight suspicious areas identified by the VLM.[30]
3. **Actionable Insights**: Include a "Recommendation" section that synthesizes the scores from all modalities into a final "Approve" or "Escalate to Investigator" decision.[4]

Phase 5: Testing, Refinement, and Pitch Prep (Hours 22–24)

The final hours should be spent on end-to-end testing and crafting the narrative.

1. **Quality Evaluation**: Use MLflow 3.0 tracing and LLM "Judges" to evaluate the accuracy of the fraud detection results against a small set of labeled data.[26]
2. **Performance Check**: Ensure the Streamlit app is responsive by using st.cache_resource for the database connections and model serving calls.[30]
3. **Pitch Deck Integration**: Capture screenshots of the Databricks lineage graph to show how the system maintains a secure audit trail—a major selling point for "real-world" regulated firms.[8]

Strategic Tips for Hackathon Success

To stand out among other teams, the implementation should address several advanced "real-world" concerns that demonstrate a deep understanding of the insurance and AI domains.

Implementation of "Agentic AI"

Instead of a single "fraud score," implement a multi-agent workflow. One agent (the "Claim Analyst") focuses on the textual narrative, while another (the "Visual Forensic Analyst") focuses on the images.[4] A third "Lead Investigator" agent then reviews both findings to issue a final verdict. This modularity makes the system easier to debug and more interpretable for human adjusters.[1]

Managing Asymmetric Risk and Accuracy

In fraud detection, the goal is to maximize the Area Under the Curve (AUC-ROC) while balancing precision and recall.[1]

$$ F_{score} = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}} $$
In a professional setting, β is often adjusted to prioritize "Recall" (identifying all potential fraud) even at the cost of some false positives, as a single undetected million-dollar fraudulent claim is more damaging than the manual review of ten legitimate ones.[1]

Security and PII Handling

For a "Chubb-ready" solution, the system must demonstrate how it handles Personally Identifiable Information (PII) like license plates or names on ID cards. The team can use ai_mask to automatically redact sensitive information in the extracted text before it is stored or shared with downstream models.[13] Furthermore, using Unity Catalog's data lineage allows for the auditing of every person or model that accessed a particular image, ensuring compliance with HIPAA or GDPR.[31]

Conclusion: A Paradigm Shift in Authentication

The proposed framework represents a shift from reactive document verification to an integrated, proactive multimodal intelligence system. By leveraging the high-performance computing and unified governance of the Databricks Intelligence Platform, organizations can build solutions that not only detect current fraud patterns but also adapt to the emerging threats of the generative AI era. The implementation of metadata forensics, narrative-visual consistency checks, and optimized video frame sampling provides a comprehensive defense against multifaceted fraud.[1]
For a 24-hour hackathon, the combination of Unity Catalog Volumes, Foundation Model APIs via ai_query, and a Streamlit-based Databricks App provides the fastest path to a high-impact, professional-grade solution. This project not only addresses a critical business problem for firms like Chubb but also demonstrates the transformative potential of the Databricks ecosystem in securing the global digital economy.[3]

Works cited

1. Multimodal AI Frameworks for Insurance Fraud Detection, accessed on February 28, 2026, https://papers.ssrn.com/sol3/Delivery.cfm/5756442.pdf?abstractid=5756442&mirid=1
2. (PDF) Multimodal AI Frameworks for Insurance Fraud Detection - ResearchGate, accessed on February 28, 2026, https://www.researchgate.net/publication/398665856_Multimodal_AI_Framework

s_for_Insurance_Fraud_Detection
3. Scaling AI: End-to-End MLOps in Databricks for Fraud Detection and Beyond - Medium, accessed on February 28, 2026, https://medium.com/@eduardodmoraes/scaling-ai-end-to-end-mlops-in-databricks-for-fraud-detection-and-beyond-e1d2e6de9919
4. Insurance claim analysis with AI using a multi-agent architecture ..., accessed on February 28, 2026, https://docs.digibee.com/documentation/resources/ai-practical-examples/insurance-claim-analysis-with-ai
5. International Journal of Multidisciplinary - Research in Science, Engineering and Technology - ijmrset, accessed on February 28, 2026, https://www.ijmrset.com/upload/221_Forensic%20Analysis.pdf
6. Distributed video processing system using Spark - Siddhant Garg, accessed on February 28, 2026, https://gargsid.github.io/data/CS532_final.pdf
7. LemurPwned/video-sampler: Effective frame sampling for ML applications. - GitHub, accessed on February 28, 2026, https://github.com/LemurPwned/video-sampler
8. Introducing Databricks Apps | Databricks Launch Blog, accessed on February 28, 2026, https://www.databricks.com/blog/introducing-databricks-apps
9. How to deploy a Streamlit app using Databricks App - Stack Overflow, accessed on February 28, 2026, https://stackoverflow.com/questions/79080228/how-to-deploy-a-streamlit-app-using-databricks-app
10. Work with unstructured data in volumes | Databricks on AWS, accessed on February 28, 2026, https://docs.databricks.com/aws/en/volumes/unstructured-data-tutorial
11. Work with files in Unity Catalog volumes | Databricks on AWS, accessed on February 28, 2026, https://docs.databricks.com/aws/en/volumes/volume-files
12. Databricks Foundation Model APIs | Databricks on AWS, accessed on February 28, 2026, https://docs.databricks.com/aws/en/machine-learning/foundation-model-apis/
13. Perform batch inference using a Spark DataFrame | Databricks on ..., accessed on February 28, 2026, https://docs.databricks.com/gcp/en/machine-learning/model-inference/dl-model-inference
14. kennethleungty/Image-Metadata-Exif: Read and modify ... - GitHub, accessed on February 28, 2026, https://github.com/kennethleungty/Image-Metadata-Exif
15. Document fraud detection software: 3 key methods - Koncile, accessed on February 28, 2026, https://www.koncile.ai/en/ressources/document-fraud-detection-software-3-methods
16. Digital image forensics using EXIF data of digital evidence | Request PDF - ResearchGate, accessed on February 28, 2026, https://www.researchgate.net/publication/359129015_Digital_image_forensics_using_EXIF_data_of_digital_evidence

17. Use foundation models | Databricks on AWS, accessed on February 28, 2026, https://docs.databricks.com/aws/en/machine-learning/model-serving/score-foundation-models
18. Use of AI-Generated Images for Fake Insurance Claims and Other Frauds | 01 - Debevoise, accessed on February 28, 2026, https://www.debevoise.com/insights/publications/2026/01/use-of-ai-generated-images-for-fake-insurance
19. How GANs and AI-Generated Images Are Transforming Insurance Fraud Detection | Innoveo, accessed on February 28, 2026, https://innoveo.com/blog/how-gans-and-ai-generated-images-are-transforming-insurance-fraud-detection/
20. Identify Suspicious Behavior in Video with Databricks Runtime for Machine Learning, accessed on February 28, 2026, https://www.databricks.com/blog/2018/09/13/identify-suspicious-behavior-in-video-with-databricks-runtime-for-machine-learning.html
21. Spark UDF — Deep Insights in Performance | by QuantumBlack, AI by McKinsey - Medium, accessed on February 28, 2026, https://medium.com/quantumblack/spark-udf-deep-insights-in-performance-f0a95a4d8c62
22. Arrow-optimized Python UDFs in Apache Spark™ 3.5 | Databricks Blog, accessed on February 28, 2026, https://www.databricks.com/blog/arrow-optimized-python-udfs-apache-sparktm-35
23. AI-Powered Claims Assessment Reference Architecture for Insurance - Databricks, accessed on February 28, 2026, https://www.databricks.com/resources/architectures/ai-powered-claims-reference-architecture-for-insurance
24. Navigating the Impact of AI in Insurance: Opportunities and Challenges | Databricks Blog, accessed on February 28, 2026, https://www.databricks.com/blog/navigating-impact-ai-insurance-opportunities-and-challenges
25. Work with unstructured data in volumes - Azure Databricks | Microsoft Learn, accessed on February 28, 2026, https://learn.microsoft.com/en-us/azure/databricks/volumes/unstructured-data-tutorial
26. Crushing your AI Hackathon on Databricks — 10 ideas - Medium, accessed on February 28, 2026, https://medium.com/@AI-on-Databricks/crushing-your-ai-hackathon-on-databricks-10-ideas-c1bffc03c0b5
27. Add a Unity Catalog volume resource to a Databricks app, accessed on February 28, 2026, https://docs.databricks.com/aws/en/dev-tools/databricks-apps/uc-volumes
28. Databricks Foundation Model APIs - Promptfoo, accessed on February 28, 2026, https://www.promptfoo.dev/docs/providers/databricks/
29. Upload a file | Databricks Apps Cookbook, accessed on February 28, 2026,

https://apps-cookbook.dev/docs/streamlit/volumes/volumes_upload/
30. Tutorial: Develop a Databricks app with Streamlit, accessed on February 28, 2026, https://docs.databricks.com/gcp/en/dev-tools/databricks-apps/tutorial-streamlit
31. Fraud Detection on Databricks: From Pilot to Always-On Production | Kriv AI, accessed on February 28, 2026, https://www.kriv.ai/articles/fraud-detection-on-databricks-from-pilot-to-always-on-production
32. Databricks Unity Catalog: A Beginner's Guide - DataCamp, accessed on February 28, 2026, https://www.datacamp.com/tutorial/databricks-unity-catalog-guide